TRISHA ACADEMY

DevOps Career Track · Student Notes



Docker Volumes & Data Persistence



Keep your data when containers come and go

Based on docs.docker.com/engine/storage/volumes · Lab: Spring Boot + H2

Named volumes **Anonymous volumes** **Bind mounts** **tmpfs** **--mount vs -v**

Read-only & subpath **Volume drivers / NFS** **Backup & restore** **H2 hands-on lab**

20 interview Q&A

For students of TRISHA ACADEMY · Keep these notes for revision and interviews

Contents

How to use these notes . . . 4

Part 1: Why containers lose data . . . 5

1.1 The writable container layer . . . 5
1.2 What is a volume? . . . 5
1.3 When to use volumes . . . 6

Part 2: Mount types . . . 7

2.1 The four mount types . . . 7
2.2 Named vs anonymous volumes . . . 8

Part 3: Volume lifecycle . . . 9

3.1 A volume outlives its containers . . . 9
3.2 Mounting a volume over existing data . . . 9

Part 4: Syntax: --mount vs -v . . . 10

4.1 Two flags, same result . . . 10
4.2 Options for --mount . . . 10
4.3 Options for -v / --volume . . . 10

Part 5: Create, inspect, use and remove . . . 12

5.1 Manage volumes . . . 12
5.2 Start a container with a volume . . . 12
5.3 Remove volumes . . . 13

Part 6: Read-only, subdirectories and pre-populating . . . 14

6.1 Populate a volume using a container . . . 14
6.2 Read-only volumes . . . 14
6.3 Mount a volume subdirectory . . . 14

Part 7: Sharing data . . . 15

7.1 Between containers on one host . . . 15
7.2 Between machines: volume drivers . . . 15

Part 8: Back up, restore and migrate . . . 16

8.1 The idea . . . 16
8.2 Back up and restore a named volume . . . 16
8.3 The official --volumes-from method . . . 16

8.4 Migrate to another server . . . 16

Part 9: Bind mounts and the VOLUME instruction . . . 18

9.1 Bind mounts . . . 18

9.2 The VOLUME instruction in a Dockerfile . . . 18

Part 10: Hands-on lab: persist the H2 database . . . 19

10.1 How the project is set up . . . 19

10.2 Build and run . . . 20

10.3 Prove that the data is kept . . . 20

10.4 Switch to a named volume . . . 20

10.5 Look at the data . . . 20

Part 11: Best practices, troubleshooting and cheat sheet . . . 22

11.1 Best practices . . . 22

11.2 Troubleshooting . . . 22

11.3 Cheat sheet . . . 23

Part 12: Top 20 interview questions . . . 24

How to use these notes

These notes teach Docker volumes from zero, following the official Docker documentation (**docs.docker.com/engine/storage/volumes**), and finish with a hands-on lab on the TRISHA ACADEMY **devops-java-demo** project - a Spring Boot Student API that stores its data in an **H2** database file.

- Read each part, then type every command yourself - do not copy-paste without reading.
- Commands are shown for **PowerShell** (Windows). Linux/macOS differences are noted where needed.
- Blue boxes quote the official documentation; green boxes are things to try; red boxes are common mistakes.
- Docker Desktop (Engine 26 or newer) is enough for every example.

Part	Topic
1	Why containers lose data, and what a volume is
2	Mount types: named, anonymous, bind, tmpfs
3	Volume lifecycle and mounting over existing data
4	Syntax: --mount vs -v, and all options
5	Create, inspect, use and remove volumes
6	Read-only volumes, subdirectories and populating volumes
7	Sharing data: between containers and with volume drivers (NFS)
8	Back up, restore and migrate
9	Bind mounts and the Dockerfile VOLUME instruction
10	Hands-on lab: persist the H2 database of devops-java-demo
11	Best practices, troubleshooting and cheat sheet
12	Top 20 interview questions

DOCKER VOLUMES · PART 01

Why containers lose data

Understand the writable layer, and why volumes are the preferred way to persist data.

1.1 The writable container layer

An image is read-only. When you start a container, Docker adds a thin **writable layer** on top of the image. Everything the application writes - log files, uploaded files, database files - goes into that layer.

Action	What happens to the writable layer
`docker stop` / `docker start` / `docker restart`	Kept - same container
`docker rm` **(delete the container)**	**Deleted** together with the container
New container from the same image	Starts with an **empty** writable layer
Rebuild the image and run again	Starts **empty** again

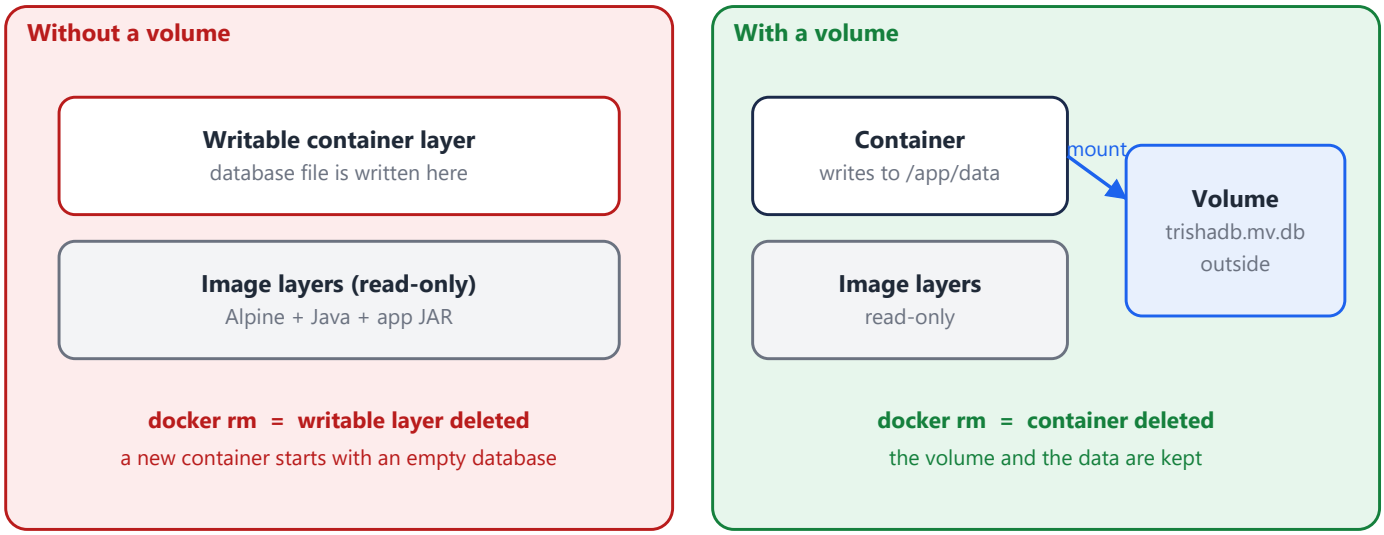


Figure 1.1 - Without a volume the data dies with the container; with a volume it lives outside

Everyday example
A container is like a hotel room. You can leave your clothes there while you stay (stop/start), but when you check out (`docker rm`) housekeeping clears the room. A volume is your own locker in the hotel lobby: you can change rooms as often as you like and your things are still in the locker.

1.2 What is a volume?

From the official Docker documentation
Volumes are persistent data stores for containers, created and managed by Docker. You can create a volume explicitly using the `docker volume create` command, or Docker can create a volume during container creation. When you create a volume, it's stored within a directory on the Docker host. When you mount the volume into a container, this directory is what's mounted into the container.

1.3 When to use volumes

The official documentation calls volumes **the preferred mechanism for persisting data** generated by and used by Docker containers, because:

- Volumes are easier to back up or migrate than bind mounts.
- You can manage them with Docker CLI commands or the Docker API.
- They work on both Linux and Windows containers.
- They can be shared more safely among multiple containers.
- New volumes can be pre-populated with content by a container or build.
- They give high-performance I/O.
- A volume does not increase the size of the container, and it is faster than writing into the writable layer (which goes through the storage driver's union filesystem).

When NOT to use a volume
Volumes are **not** a good choice when you need to open the files directly from the host (for example in Windows Explorer or your IDE). For that, use a **bind mount** (Part 9). For temporary data that must never be stored, use a **tmpfs** mount.

DOCKER VOLUMES · PART 02

Mount types

Know the four ways to give a container storage, and pick the right one.

2.1 The four mount types

Type	Example	Where data lives	Use it for
Named volume	`-v trishadb-data:/app/data`	Docker storage area, chosen name	Databases, production data
Anonymous volume	`-v /app/data` or `VOLUME` in Dockerfile	Docker storage area, random name	Short-lived data
Bind mount	`-v "${PWD}\data:/app/data"`	Any host folder you choose	Development, demos, config files
tmpfs	`--tmpfs /app/tmp`	Host memory (RAM) only	Secrets, scratch files (Linux)

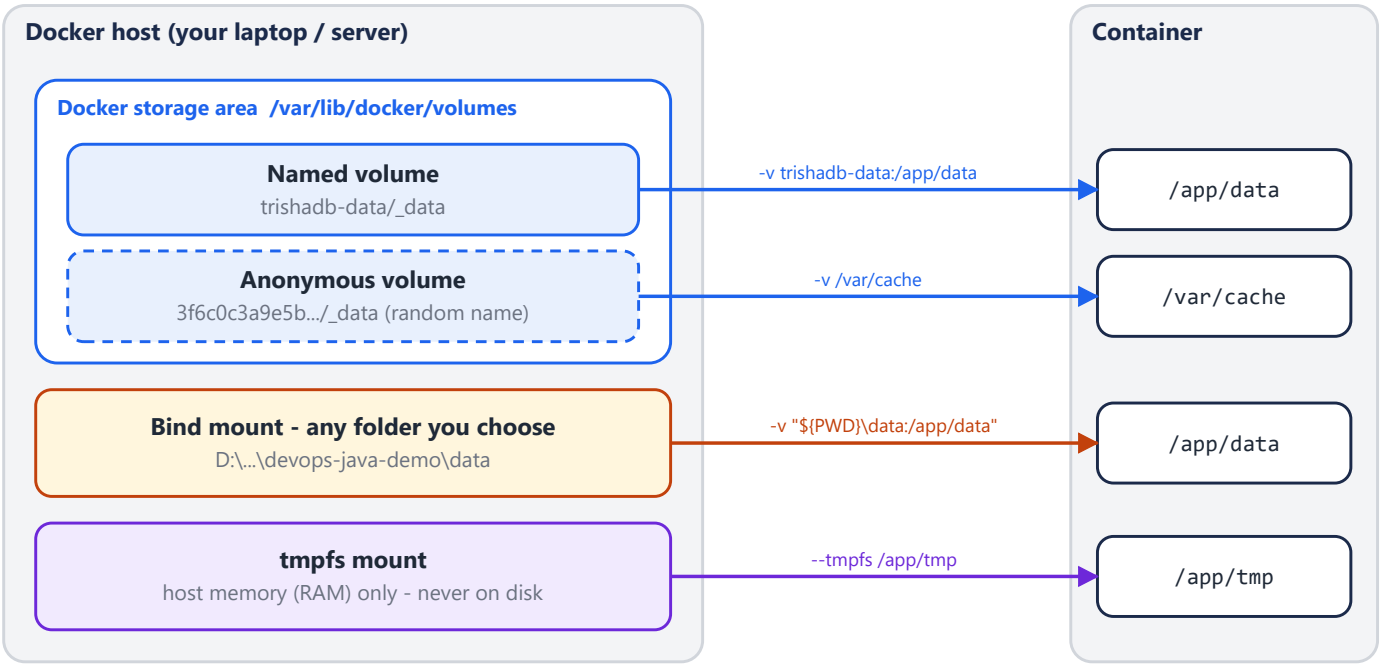


Figure 2.1 - Where each mount type stores its data

Did you know?
Docker also has **image mounts** (mount the content of another image read-only into a container) - see the 'Next steps' links on the official page.

2.2 Named vs anonymous volumes

From the official Docker documentation

- A volume may be named or anonymous. Anonymous volumes are given a random name that's guaranteed to be unique within a given Docker host.
- Just like named volumes, anonymous volumes persist even if you remove the container that uses them, except if you use the `--rm` flag when creating the container.
- If you create multiple containers consecutively that each use anonymous volumes, each container creates its own volume. Anonymous volumes aren't reused or shared between containers automatically.

Common mistake

Running `docker run` without a named `-v` and expecting the data back in the next container. Each new container gets a **new** anonymous volume, so the database looks empty - the old data is still in an orphaned anonymous volume that nobody uses.

DOCKER VOLUMES · PART 03

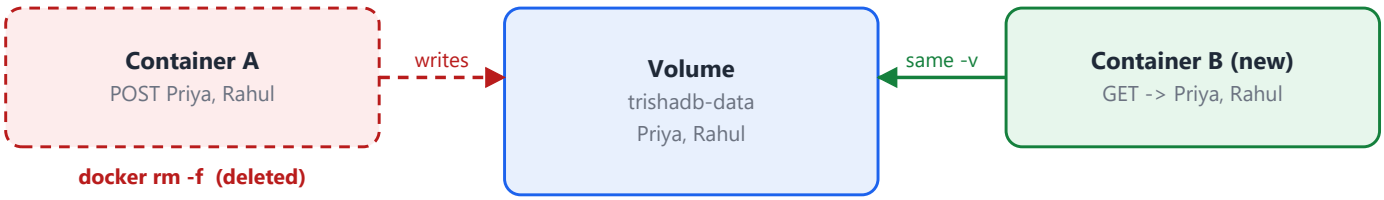
Volume lifecycle

See how long a volume lives, and what happens when it is mounted over existing files.

3.1 A volume outlives its containers

From the official Docker documentation

A volume's contents exist outside the lifecycle of a given container. When a container is destroyed, the writable layer is destroyed with it. Using a volume ensures that the data is persisted even if the container using it is removed. A given volume can be mounted into multiple containers simultaneously. When no running container is using a volume, the volume is still available to Docker and isn't removed automatically.



The volume lives outside every container: delete, recreate, rebuild - the data stays until you delete the volume itself.

Figure 3.1 - Delete container A, start container B with the same volume: the data is still there

3.2 Mounting a volume over existing data

Situation	What Docker does
Non-empty volume mounted on a folder that has files in the image	The image's files are **hidden** (obscured) by the volume's files - like plugging a USB drive into `/mnt`. To see them again you must recreate the container without the mount.
Empty volume mounted on a folder that has files in the image	The image's files are **copied into the volume** first (default behaviour).
Volume name that does not exist yet	Docker creates an empty volume for you.
Empty volume + `volume-nocopy` option	Nothing is copied - the volume stays empty.

Remember

Pre-populating is a handy way to give another container the data it needs: the first container fills the new volume from its image, later containers just mount it.

DOCKER VOLUMES · PART 04

Syntax: --mount vs -v

Read and write both flags, and know every option.

4.1 Two flags, same result

Syntax

```
docker run --mount type=volume,src=<volume-name>,dst=<mount-path>
docker run --volume <volume-name>:<mount-path>      # short form: -v
```

From the official Docker documentation

In general, `--mount` is preferred. The main difference is that the `--mount` flag is more explicit and supports all the available options. You must use `--mount` to specify volume driver options or to mount a volume subdirectory.

4.2 Options for --mount

Comma-separated `key=value` pairs; the order does not matter.

Option	Meaning
`source`, `src`	Name of the volume. Omit it for an anonymous volume.
`destination`, `dst`, `target`	Path inside the container.
`readonly`, `ro`	Mount read-only.
`volume-nocopy`	Do not copy the image's files into an empty volume.
`volume-subpath`	Mount only a subdirectory of the volume (it must already exist).
`volume-opt`	Driver option as key=value; can be given more than once.
`volume-driver`	Use a specific volume driver (default `local`).

Example

```
docker run --mount type=volume,src=myvolume,dst=/data,ro,volume-subpath=/foo alpine ls /data
```

4.3 Options for -v / --volume

Three fields separated by colons, in this exact order: `[<volume-name>:]<mount-path>[:opts]`. Leave out the name for an anonymous volume.

Option (3rd field)	Meaning
`readonly`, `ro`	Mount read-only.
`volume-nocopy`	Do not copy the image's files into an empty volume.

Goal	-v	--mount
Named volume	`-v trishadb-data:/app/data`	`--mount source=trishadb-data,target=/app/data`

Goal	-v	--mount
Anonymous volume	`-v /app/data`	`--mount type=volume,target=/app/data`
Read-only	`-v trishadb-data:/app/data:ro`	`--mount source=trishadb-data,target=/app/data,ro`
Bind mount	`-v "${PWD}\data:/app/data"`	`--mount type=bind,src="${PWD}\data",dst=/app/data`
tmpfs	`--tmpfs /app/tmp`	`--mount type=tmpfs,dst=/app/tmp`

Common mistake
With `-v`, a typo in a bind-mount path silently creates a new empty folder. `--mount type=bind` stops with an error if the source folder does not exist - safer in scripts.

DOCKER VOLUMES · PART 05

Create, inspect, use and remove

Manage volumes with the Docker CLI and verify mounts.

5.1 Manage volumes

Unlike a bind mount, a volume can be created and managed outside the scope of any container.

PowerShell

```
docker volume create trishadb-data       # create
docker volume ls                         # list
docker volume inspect trishadb-data      # details
docker volume rm trishadb-data           # remove (must not be used by any container)
```

Output of docker volume inspect

```
[
  {
    "Driver": "local",
    "Labels": {},
    "Mountpoint": "/var/lib/docker/volumes/trishadb-data/_data",
    "Name": "trishadb-data",
    "Options": {},
    "Scope": "local"
  }
]
```

Did you know?

On Docker Desktop (Windows/macOS) the `Mountpoint` is inside Docker Desktop's Linux VM, not a normal Windows folder. To look inside a volume, use a throw-away container: `docker run --rm -v trishadb-data:/data alpine ls -l /data`

5.2 Start a container with a volume

If the volume does not exist yet, Docker creates it. Both commands below give the same result.

PowerShell (` continues the line)

```
docker run -d --name devops-java-demo -p 9090:9090 `
  --mount source=trishadb-data,target=/app/data devops-java-demo:1.0.0

docker run -d --name devops-java-demo -p 9090:9090 `
  -v trishadb-data:/app/data devops-java-demo:1.0.0
```

Verify the mount with `docker inspect devops-java-demo` and look at the `Mounts` section:

Check the mount

```
docker inspect --format "{{json .Mounts}}" devops-java-demo

[{"Type":"volume","Name":"trishadb-data",
  "Source":"/var/lib/docker/volumes/trishadb-data/_data",
  "Destination":"/app/data","Driver":"local","Mode":"z","RW":true,"Propagation":""}]
```

Field	Meaning
`Type`	`volume`, `bind` or `tmpfs`
`Name` / `Source`	Volume name and where Docker stores it
`Destination`	Path inside the container
`RW`	`true` = read-write, `false` = read-only

5.3 Remove volumes

Command	What it removes
`docker volume rm <name>`	One volume (fails if a container, even a stopped one, uses it)
`docker run --rm -v /foo -v awesome:/bar busybox top`	With `--rm`, the anonymous `/foo` volume is removed when the container exits; the named `awesome` volume is kept
`docker rm -v <container>`	The container and its **anonymous** volumes (never named ones)
`docker volume prune`	All unused **anonymous** volumes (Docker Engine 23 and newer)
`docker volume prune -a`	All unused volumes, **named ones too**

Common mistake
`docker volume prune -a` and `docker volume rm` delete data for good - there is no recycle bin. Take a backup first (Part 8).

Did you know?
If another container mounted the volumes with `--volumes-from`, the volume definitions are copied, and the anonymous volume stays after the first container is removed.

DOCKER VOLUMES · PART 06

Read-only, subdirectories and pre-populating

Control how a container can use a volume.

6.1 Populate a volume using a container

When a container creates a **new** volume on a folder that has files in the image, Docker copies those files into the volume. Here the `nginx-vol` volume is filled with Nginx's default web page:

PowerShell

```
docker run -d --name=nginxtest -v nginx-vol:/usr/share/nginx/html nginx:latest
docker run --rm -v nginx-vol:/data alpine ls /data      # 50x.html  index.html
docker rm -f nginxtest
docker volume rm nginx-vol
```

6.2 Read-only volumes

Several containers can mount the same volume - some read-write, others read-only. Add `ro` (or `readonly` with `--mount`):

PowerShell

```
docker run -d --name=nginxtest -v nginx-vol:/usr/share/nginx/html:ro nginx:latest
docker run -d --name=nginxtest2 `
  --mount source=nginx-vol,destination=/usr/share/nginx/html,readonly nginx:latest
```

`docker inspect` then shows `"RW": false`. Any write inside the mount fails with *Read-only file system*.

Common mistake
Never mount a **database** folder read-only - H2, MySQL and PostgreSQL must write to it.

6.3 Mount a volume subdirectory

With `volume-subpath` (only with `--mount`) each container sees just one folder of a shared volume. The subdirectory must exist before you mount it.

Separate log folders in one volume

```
docker volume create logs
docker run --rm --mount src=logs,dst=/logs alpine mkdir -p /logs/app1 /logs/app2
docker run -d --name=app1 --mount src=logs,dst=/var/log/app1,volume-subpath=app1 app1:latest
docker run -d --name=app2 --mount src=logs,dst=/var/log/app2,volume-subpath=app2 app2:latest
```

Each container writes to its own subfolder of the `logs` volume and cannot see the other one's logs.

DOCKER VOLUMES · PART 07

Sharing data

Share volumes between containers and between machines.

7.1 Between containers on one host

PowerShell

```
# The app writes to the volume
docker run -d --name devops-java-demo -p 9090:9090 -v trishadb-data:/app/data devops-java-demo:1.0.0
# Another container reads the same volume
docker run --rm -v trishadb-data:/data alpine ls -l /data
# Or reuse every volume of an existing container
docker run --rm --volumes-from devops-java-demo alpine ls -l /app/data
```

Common mistake

H2 in file mode allows only **one** application to open the database at a time. Sharing the volume for reading, inspection or backup is fine - two app containers on one H2 file is not.

7.2 Between machines: volume drivers

For fault-tolerant apps with replicas on several servers, either store files in cloud object storage (for example Amazon S3) from the application, or create volumes with a driver that writes to external storage (NFS, SFTP, S3 ...). The application code does not change.

NFS volume with the built-in local driver (Linux host)

```
docker volume create --driver local `
  --opt type=nfs `
  --opt o=addr=10.0.0.10,rw,nfsvers=4 `
  --opt device=:/var/docker-nfs `
  nfsvolume
docker run -d -v nfsvolume:/app nginx:latest
```

Third-party driver plugin (SFTP via rclone)

```
docker plugin install --grant-all-permissions rclone/docker-volume-rclone --aliases rclone
docker volume create -d rclone --name rclonevolume `
  -o type=sftp -o path=remote -o sftp-host=1.2.3.4 -o sftp-user=user
```

Driver / option	Stores data on
`local` **(default)**	The Docker host's disk
`local` + `type=nfs`	An NFS server (NFSv3 / NFSv4)
`local` + `type=cifs`	A Windows / Samba share
`rclone` **plugin**	SFTP, S3 and many cloud stores
Cloud CSI / plugins	Amazon EBS / EFS, Azure Disk / Files, etc.

Did you know?

If a driver option itself contains commas (like NFS `o=` options) inside `--mount`, put that option in double quotes and the whole `--mount` value in single quotes.

DOCKER VOLUMES · PART 08

Back up, restore and migrate

Protect volume data and move it between hosts.

8.1 The idea

Start a short-lived helper container that mounts **both** the volume and a host folder, and use `tar` to copy between them. Stop the application first so the files are not changing.



Restore = the same idea in reverse (tar xzf into an empty volume)

Figure 8.1 - A temporary container archives the volume into a host folder

8.2 Back up and restore a named volume

PowerShell

```
docker stop devops-java-demo

# Backup  ->  .\trishadb-backup.tar.gz
docker run --rm -v trishadb-data:/data -v "${PWD}:/backup" alpine `
  tar czf /backup/trishadb-backup.tar.gz -C /data .

# Restore into a new volume and start a second copy of the app on port 9091
docker volume create trishadb-data-restored
docker run --rm -v trishadb-data-restored:/data -v "${PWD}:/backup" alpine `
  tar xzf /backup/trishadb-backup.tar.gz -C /data
docker run -d --name devops-java-demo-restored -p 9091:9090 `
  -v trishadb-data-restored:/app/data devops-java-demo:1.0.0
```

8.3 The official --volumes-from method

Back up and restore via another container

```
docker run -v /dbdata --name dbstore ubuntu /bin/bash
docker run --rm --volumes-from dbstore -v "${PWD}:/backup" ubuntu tar cvf /backup/backup.tar /dbdata

docker run -v /dbdata --name dbstore2 ubuntu /bin/bash
docker run --rm --volumes-from dbstore2 -v "${PWD}:/backup" ubuntu `
  bash -c "cd /dbdata && tar xvf /backup/backup.tar --strip 1"
```

8.4 Migrate to another server

- Back up the volume to a `.tar.gz` file (8.2).
- Copy the file to the new server (`scp`, S3, USB ...).
- Create a volume there, restore into it, and start the container with the same `-v`.

Try it yourself
Back up your `trishadb-data` volume, delete the volume, restore it from the archive and check that `GET /students` still returns your students.

DOCKER VOLUMES · PART 09

Bind mounts and the VOLUME instruction

Compare bind mounts with volumes, and see what `VOLUME` in a Dockerfile really does.

9.1 Bind mounts

A bind mount maps a folder that **you** choose on the host into the container. The files are normal host files - you can open, edit, copy and back them up directly.

Bind mount

```
# PowerShell - run from the project folder
docker run -d --name devops-java-demo -p 9090:9090 -v "${PWD}\data:/app/data" devops-java-demo:1.0.0
# Git Bash / Linux / macOS
docker run -d --name devops-java-demo -p 9090:9090 -v "$(pwd)/data:/app/data" devops-java-demo:1.0.0
```

	Volume	Bind mount
Managed by	Docker	You (host folder)
See files in Explorer / IDE	No (inside Docker's storage)	Yes
Depends on host folder layout and OS	No	Yes
Copies image files into an empty mount	Yes (unless volume-nocopy)	No - hides the image's files
Speed on Docker Desktop (Windows/macOS)	Fast	Slower (files cross into the VM)
Backup / migrate	Docker CLI (Part 8)	Copy the folder
Best for	Production, servers	Development, demos, config

9.2 The VOLUME instruction in a Dockerfile

Dockerfile of devops-java-demo

```
WORKDIR /app
VOLUME ["/app/data"]
```

- Marks `/app/data` as a folder whose data belongs **outside** the container.
- Run **without** `-v`: Docker creates an anonymous volume for `/app/data` automatically.
- Run **with** `-v`: your named volume or bind mount is used instead.
- A Dockerfile can **never** choose a host folder or a volume name - only `docker run` can.

Common mistake

Relying on `VOLUME` alone. A new container gets a new anonymous volume, so the data seems to disappear. Always run with a named volume or a bind mount.

DOCKER VOLUMES · PART 10

Hands-on lab: persist the H2 database

Make the devops-java-demo Student API keep its data through restarts, deletes, hard kills and rebuilds.

10.1 How the project is set up

File	Setting	Why
`application.properties`	`spring.datasource.url=jdbc:h2:file:./data/trishadb;WRITE_DELAY=0`	H2 writes to the file `data/trishadb.mv.db` instead of memory
`application.properties`	`spring.jpa.hibernate.ddl-auto=update`	Keeps tables and rows on restart (`create` would wipe them)
`application.properties`	`spring.h2.console.settings.web-allow-others=true`	Lets the H2 console open from your browser when the app runs in Docker
`Dockerfile`	`WORKDIR /app` + `VOLUME ["/app/data"]`	`./data` inside the container = `/app/data`
`.dockerignore`	`data/`	Your local database is never copied into the image
`pom.xml` **(Surefire)**	`jdbc:h2:mem:testdb`	JUnit tests use a throw-away in-memory database

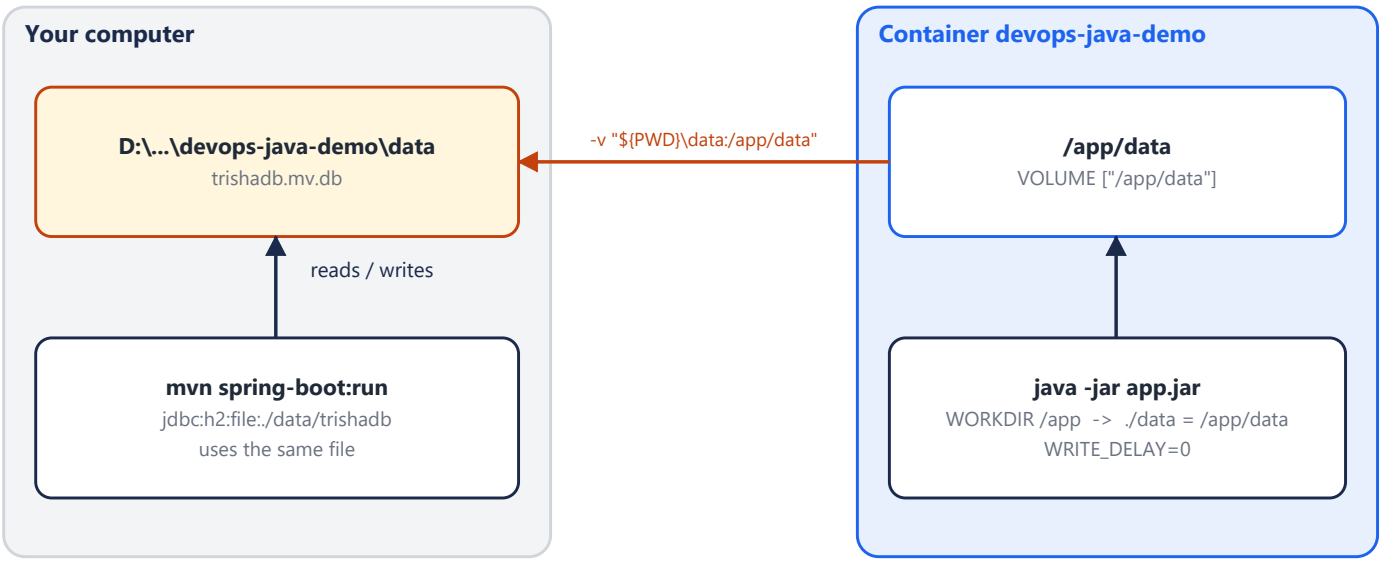


Figure 10.1 - The project's data folder is bind-mounted to /app/data; mvn spring-boot:run uses the same file

Why WRITE_DELAY=0?
By default H2 keeps the newest changes in memory for up to about a second before writing them. In our tests, a student saved just before `docker rm -f` was lost. With `WRITE_DELAY=0` every change is written at once, and the same test kept all data.

10.2 Build and run

PowerShell

```
cd D:\TRISHA-ACADEMY\DevOPs\devops-java-demo
docker build -t devops-java-demo:1.0.0 .
docker run -d --name devops-java-demo -p 9090:9090 -v "${PWD}\data:/app/data" devops-java-demo:1.0.0
docker logs -f devops-java-demo        # wait for "Started SpringLabApplication", then Ctrl + C
```

Helper commands: add and list students

```
Invoke-RestMethod -Method Post -Uri http://localhost:9090/students `
  -ContentType "application/json" -Body '{"name":"Priya","course":"DevOps"}'
Invoke-RestMethod http://localhost:9090/students
```

10.3 Prove that the data is kept

#	Test	Commands	Expected
1	File in project	Add 2 students, then `Get-ChildItem data`	`trishadb.mv.db` exists
2	Restart	`docker restart devops-java-demo`	Students still listed
3	Delete container	`docker rm -f devops-java-demo`, then the same `docker run ... -v ...`	Students still listed
4	Hard kill	Add a student, `docker kill`, `docker start`	New student kept (ids may jump)
5	Rebuild image	`docker rm -f`, `docker build`, `docker run ... -v ...`	Students still listed
6	Local run	`docker stop`, then `mvn spring-boot:run`	Same students
7	No volume	Run without `-v`, add, `docker rm -f -v`, run again	**Empty** list
8	Tests isolated	`mvn test`, compare file time before/after	File unchanged

Did you know?
After a hard kill the next id may jump (1, 2, 33 ...). H2 reserves ids in blocks of 32 and skips the unused ones after a crash. No data is lost.

10.4 Switch to a named volume

PowerShell

```
docker rm -f devops-java-demo
docker run -d --name devops-java-demo -p 9090:9090 -v trishadb-data:/app/data devops-java-demo:1.0.0
```

Now Docker stores the database in the `trishadb-data` volume instead of the project folder - this is how you would run it on a server.

10.5 Look at the data

- H2 console: http://localhost:9090/h2-console - JDBC URL `jdbc:h2:file:./data/trishadb`, user `sa`, password `sa@123`, then `SELECT * FROM students;`
- Bind mount: open `devops-java-demo\data` in Explorer.
- Named volume: `docker run --rm -v trishadb-data:/data alpine ls -l /data`
- Reset: stop the app and delete the `data` folder (or `docker volume rm trishadb-data`).

Try it yourself
Full step-by-step test scripts with expected output are in `DB-PERSISTENCE-TESTING.md` in the project folder.

DOCKER VOLUMES · PART 11

Best practices, troubleshooting and cheat sheet

Avoid the classic mistakes.

11.1 Best practices

- Never keep important data in the container's writable layer.
- Use **named volumes** on servers and in production; **bind mounts** for development and demos.
- Prefer `--mount` in scripts - it is explicit and fails on wrong bind paths.
- Give volumes clear names; avoid relying on anonymous volumes.
- Keep data folders out of the image with `.dockerignore`.
- One writer per database file; never run two app containers on the same H2 file.
- Stop the app before a backup, and test your restore regularly.
- For production databases use the official image with a named volume, e.g. `-v pgdata:/var/lib/postgresql/data`, or a managed database such as Amazon RDS.

11.2 Troubleshooting

Problem	Cause and fix
Data gone after `docker rm` + `docker run`	`-v` missing or a different name/folder - use exactly the same `-v` every time
New container empty although `data\trishadb.mv.db` **exists**	Command run from another folder, so `${PWD}` points elsewhere - `cd` first
`Database may be already in use`	Two apps use the same H2 file - stop one
`volume is in use`	A (stopped) container uses it - `docker ps -a --filter volume=<name>`
Many unknown volumes	Anonymous volumes from runs without `-v` - `docker volume prune`
H2 console: remote connections not allowed	Set `spring.h2.console.settings.web-allow-others=true`
`Read-only file system`	Volume mounted with `ro` / `readonly`
Bind mount fails on Windows	Use a full path (`${PWD}\data`) and check Docker Desktop file sharing

11.3 Cheat sheet

Goal	Command
Create / list / inspect	`docker volume create v` · `docker volume ls` · `docker volume inspect v`
Run with a named volume	`docker run -v v:/path image` or `--mount source=v,target=/path`
Run with a bind mount	`docker run -v "${PWD}\data:/path" image`
Read-only	`-v v:/path:ro` or `--mount source=v,target=/path,readonly`
Which containers use v	`docker ps -a --filter volume=v`
Mounts of a container	`docker inspect --format "{{json .Mounts}}" c`
Look inside	`docker run --rm -v v:/data alpine ls -l /data`
Remove	`docker volume rm v` · `docker rm -v c` · `docker volume prune [-a]`
Disk usage	`docker system df -v`
Backup	`docker run --rm -v v:/data -v "${PWD}:/backup" alpine tar czf /backup/v.tgz -C /data .`

DOCKER VOLUMES · PART 12

Top 20 interview questions

Answer aloud before reading the model answer.

Interviewers look for a clear definition, one real example and one trade-off.

Q1. Why is data lost when a container is deleted?

It is written into the container's writable layer, which is destroyed with the container. A new container from the same image starts with an empty writable layer.

Q2. What is a Docker volume?

Persistent storage created and managed by Docker, stored in a directory on the host and mounted into containers. It lives independently of any container.

Q3. Why are volumes preferred over writing to the container?

They persist after the container is removed, do not grow the container size, are faster (no union filesystem / storage driver), and are easy to back up, migrate and share.

Q4. Named vs anonymous volume?

A named volume has a name you choose and can be reused by any container. An anonymous volume gets a random name; each new container creates its own, so it is not reused automatically.

Q5. Volume vs bind mount?

A volume is managed by Docker in its storage area and is independent of the host layout. A bind mount maps a host folder you choose - good when you need the files on the host, but it depends on the host's paths and OS.

Q6. When would you use tmpfs?

For non-persistent or sensitive data that must never be written to disk, and to avoid writing into the writable layer. It lives in RAM and disappears when the container stops.

Q7. --mount or -v - which is preferred and why?

--mount: it is explicit key=value syntax and supports all options. It is required for volume driver options and volume-subpath.

Q8. What happens when you mount an empty volume over a folder that has files?

Docker copies the existing files into the volume first, unless the volume-nocopy option is used.

Q9. And a non-empty volume?

The existing files in the container are obscured (hidden) by the volume's content, like mounting a USB drive over /mnt.

Q10. What does the VOLUME instruction in a Dockerfile do?

It marks a path as external storage. If no -v is given, Docker creates an anonymous volume for it. It cannot specify a host path or a volume name.

Q11. Does docker rm -v delete named volumes?

No. It removes only the anonymous volumes associated with the container.

Q12. What does --rm do to volumes?

When the container exits it is removed together with its anonymous volumes; named volumes are kept.

Q13. docker volume prune vs docker volume prune -a?

On Docker Engine 23+, prune removes unused anonymous volumes; -a removes all unused volumes including named ones. Both are irreversible.

Q14. How do you back up a volume?

Run a temporary container that mounts the volume and a host folder (or uses --volumes-from) and archive the volume with tar into the host folder. Stop the writing application first.

Q15. How do you check which volume a container uses?

docker inspect <container> and read the Mounts section (Type, Name, Source, Destination, RW).

Q16. How do you share data between containers?

Mount the same named volume in each, or use --volumes-from. Several containers can mount a volume at the same time, some read-write and some read-only.

Q17. What is volume-subpath used for?

Mounting only one subdirectory of a volume into a container, e.g. separate log folders per app in one shared logs volume. The subdirectory must already exist.

Q18. How can replicas on different servers share data?

Store files in object storage from the app (e.g. S3), or use a volume driver backed by shared storage such as NFS, CIFS, SFTP (rclone) or cloud file systems.

Q19. How do you keep an H2 database in Docker?

Use H2 file mode (jdbc:h2:file:...), ddl-auto=update, and mount a named volume or bind mount on the data folder. Add WRITE_DELAY=0 so a hard kill cannot lose the last writes.

Q20. Where is a volume stored on Docker Desktop?

Inside Docker Desktop's Linux VM at /var/lib/docker/volumes/<name>/_data, not in a Windows folder. To see the files, run a throw-away container: docker run --rm -v <name>:/data alpine ls -l /data.

Reference

- Docker docs - Volumes: https://docs.docker.com/engine/storage/volumes/
- Docker docs - Bind mounts: https://docs.docker.com/engine/storage/bind-mounts/
- Docker docs - tmpfs mounts: https://docs.docker.com/engine/storage/tmpfs/
- Project notes: README.md, DOCKER-VOLUMES.md, DB-PERSISTENCE-TESTING.md